

Class 04 | 02 August

We will start sharp at 09.05 PM

Understanding the .NET Ecosystem

The managed runtime, the CLR, the SDK — and everything you can build on top of it.

Segment 00

Learning Checkpoint (Class 1 - 3)

Verify PR before submit

- **Check the from and to branch.**
 - If you have confusion there, then see our git and github videos and Recording of class 1 (after 1 hour 300 minutes)
- **Review the Files changed tab** to ensure only the intended changes are included.
 - The last tab of the PR is the file changed.

Importance of .gitignore

- Prevents unnecessary files (e.g., .vs, bin, obj) from being tracked by Git.

Communication

- Communication is one of the most important skills in a corporate environment.
 - Before asking a question, complete the assigned classes and watch the recorded videos. Many questions are already answered there.
 - Know when to be proactive and when to be reactive:
 - Be **proactive** by communicating early, asking for clarification when needed, and informing the team about potential risks.
 - Be **reactive** by responding promptly to issues, feedback, and unexpected situations.
-

Today's agenda

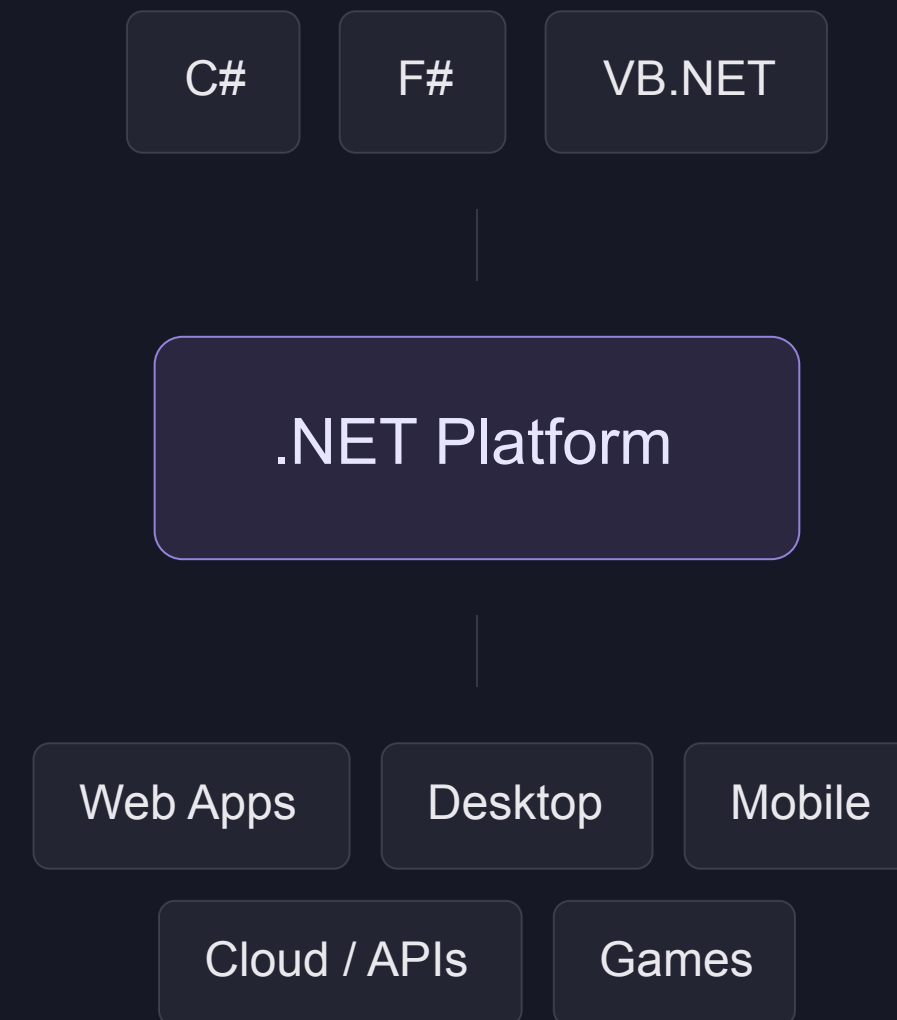
- 1 · Introduction to .NET
- 2 · Clearing the Confusion
- 3 · Core Components of .NET
- 4 · Types of .NET Applications
- 5 · Q&A + Wrap-up

Segment 01

Introduction to .NET

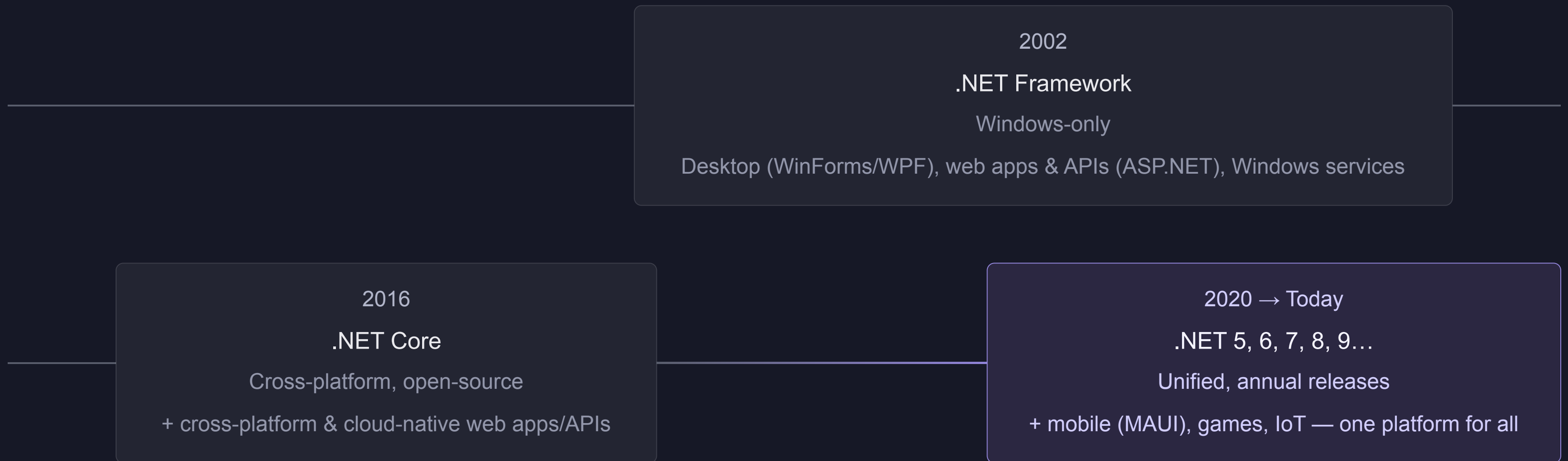
What is .NET?

- A free, open-source **developer platform** — not a language
- Runs languages like C#, F#, and VB.NET
- Includes a runtime, libraries, and tools
- Builds web, desktop, mobile, cloud, games, and IoT apps
- Maintained by Microsoft and the open-source community



A Brief History of .NET

Two roads that became one highway — most naming confusion in .NET traces back to this timeline.



- Framework is now in maintenance mode — all new work happens on modern .NET
- No ".NET 4" after Core — Microsoft skipped to 5 so it wouldn't be confused with Framework 4.x
- Mobile followed its own path: Xamarin (2011, built on Mono) joined Microsoft in 2016, and its successor .NET MAUI shipped as a first-class part of unified .NET in .NET 6

Why .NET Matters Today



Cross-Platform

Same code runs on Windows, macOS, and Linux



Open Source

Runtime and libraries live on GitHub, fully transparent



Performance

Ranks among the fastest web framework benchmarks



Security

Memory-safe runtime, regular CVE patching, active LTS support

Backed by Microsoft, strong LTS support, and a healthy job market.

Segment 02

Clearing the Confusion

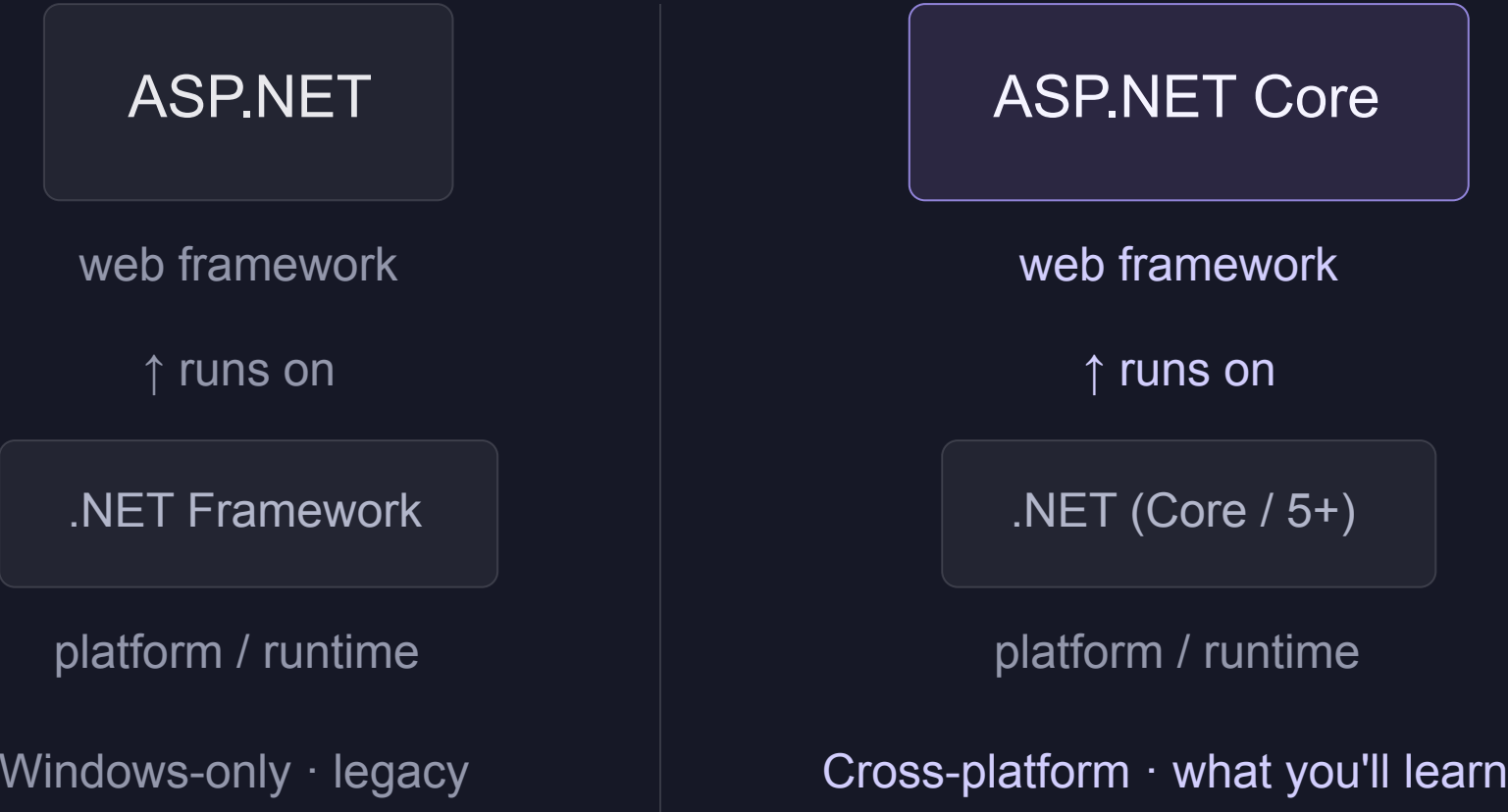
Three Confusing Names, Untangled

NAME	WHAT IT IS	STATUS
.NET Framework	Original runtime, Windows-only	Legacy
.NET Core / .NET 5+	Modern, cross-platform runtime	Active
.NET Standard	Compatibility contract for libraries	Superseded

Rule of thumb: for new projects, target modern .NET.

ASP.NET vs ASP.NET Core

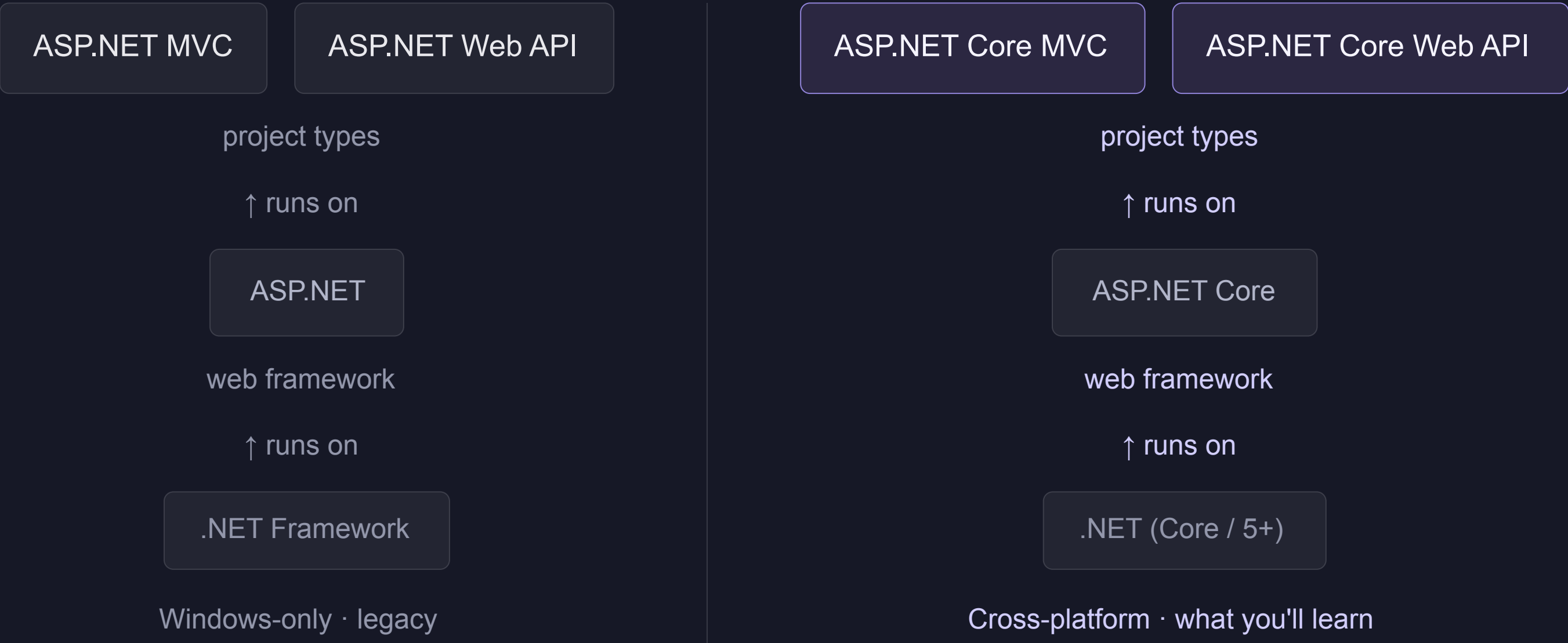
The platform is the foundation; ASP.NET is the **web framework that runs on top of it** — you don't build web apps directly in ".NET Framework", you build them in ASP.NET, which needs .NET Framework underneath to run.



Same pairing, same rule: ASP.NET Core is the web framework; .NET Core / .NET 5+ is what it runs on. It also includes Web API, MVC, Razor Pages, and Blazor.

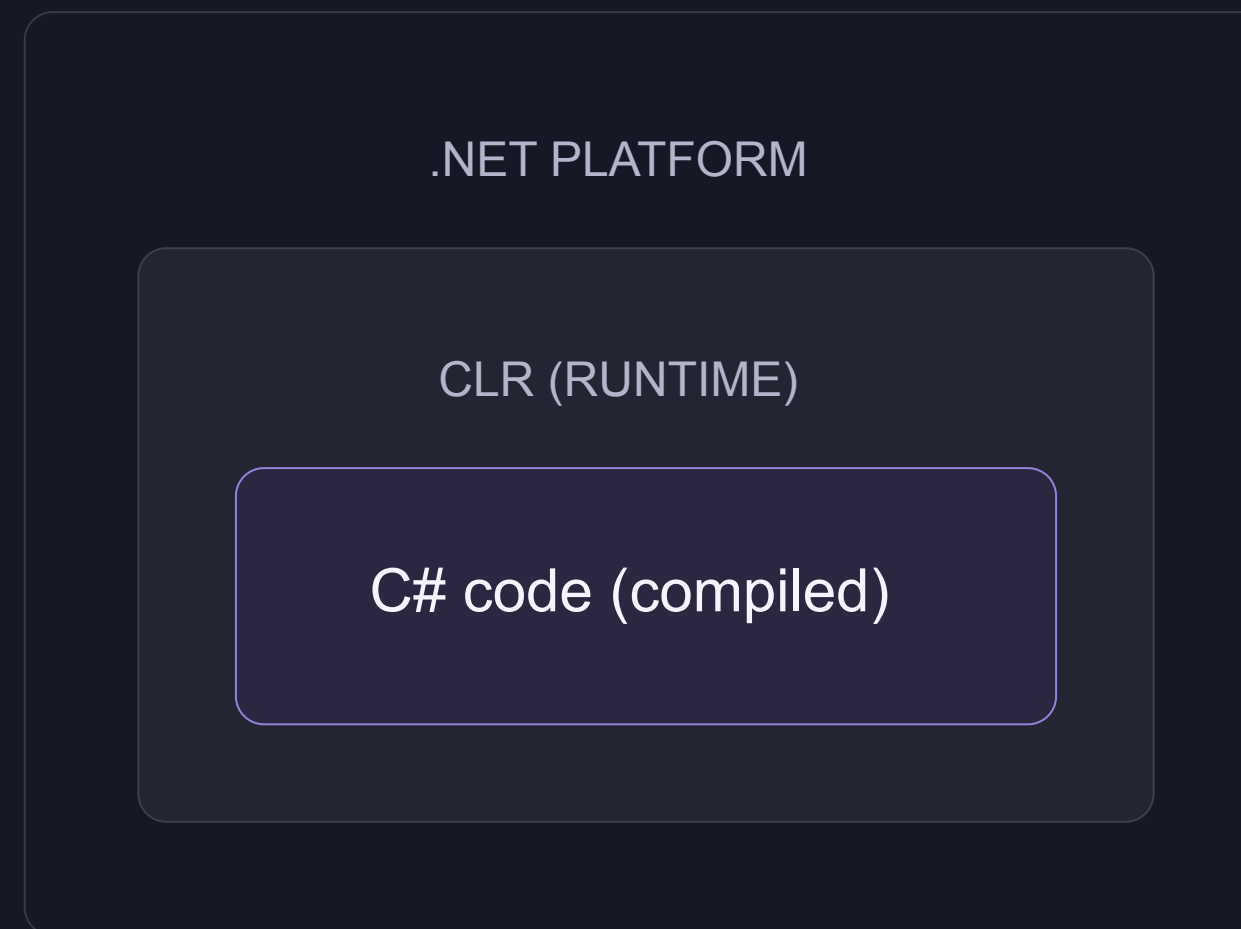
MVC & Web API: Project Types, Not Platforms

MVC and Web API aren't separate technologies — they're **project types (templates) built on top of** ASP.NET or ASP.NET Core, using the same "runs on" rule as before.



MVC returns HTML views (full pages) · Web API returns raw data (usually JSON) · Core versions are cross-platform, the originals are Windows-only legacy.

Language vs Runtime vs Platform

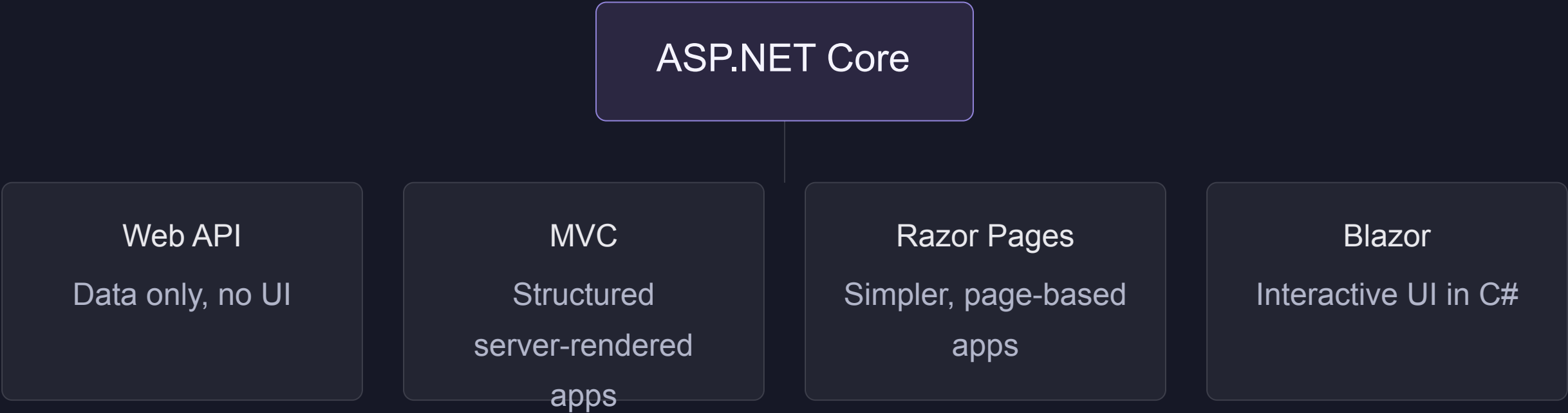


C# — the language you write

CLR — the runtime engine that executes it

.NET — the platform bundling it all together

Where Do These Fit?



Why Not Just Use the .NET Platform Directly?

The .NET platform runs any C# code, but it has **no idea what an HTTP request is** . A web framework is the layer that adds web-specific building blocks on top — so you don't write that plumbing yourself.

.NET Platform Alone

- ✓ Runs any compiled C# code
- ✓ Console apps, class libraries
- ✗ No HTTP server
- ✗ No routing or middleware
- ✗ No request/response handling

You'd have to build all of this yourself

.NET Platform + ASP.NET Core

- ✓ Built-in HTTP server (Kestrel)
- ✓ Routing & middleware pipeline
- ✓ Model binding & validation
- ✓ Dependency injection for web apps
- ✓ Everything the platform already gives you, plus this

Web-specific building blocks, ready to use

Same reason a desktop app uses WinForms/WPF instead of raw .NET: the platform is general-purpose, the framework specializes it for one kind of app.

The Big Picture — All Terms in One Table (1/2)

"Project type" below means the kind of application you're building — console app, web API, desktop app, and so on.

TERM	CATEGORY	RUNS ON	ONE-LINE DESCRIPTION
.NET	Platform	—	The overall ecosystem — runtime, libraries, tools
C#	Language	CLR / .NET	The code you write
CLR	Runtime	Part of .NET	Executes your compiled code
.NET Framework	Legacy platform	—	Old, Windows-only version of .NET
.NET Standard	Specification	—	Old compatibility contract for libraries
ASP.NET	Legacy web framework	.NET Framework	Old, Windows-only way to build web apps/APIs
ASP.NET Core	Web framework	.NET (Core / 5+)	Builds web apps/APIs on modern .NET
Web API	Project type	ASP.NET Core	Builds APIs that return data (JSON)
MVC	Project type	ASP.NET Core	Structured, server-rendered web apps

The Big Picture — All Terms in One Table (2/2)

Continued: the remaining project types you can build on top of the platform.

TERM	CATEGORY	RUNS ON	ONE-LINE DESCRIPTION
Razor Pages	Project type	ASP.NET Core	Simpler server-rendered web pages
Blazor	Project type	ASP.NET Core	Interactive web UI written in C#
Console Application	Project type	.NET (no framework)	Runs directly on the platform, no web framework needed
Class Library	Project type	.NET (no framework)	Reusable code shared across other projects
WinForms / WPF	Project type	.NET Framework or .NET	Windows-only desktop apps
.NET MAUI	Project type	.NET (Core / 5+)	Cross-platform apps, successor to Xamarin
Worker Service	Project type	.NET (no framework)	Long-running background process

Segment 03

Core Components of .NET

Six Pieces That Make .NET Work

01

CLR

The runtime engine that executes your code

02

Garbage Collector

Automatic memory cleanup, so you don't manage it by hand

03

JIT Compiler

Translates IL code into native machine code at run time

04

BCL

The built-in library of ready-to-use classes and functions

05

CTS & CLS

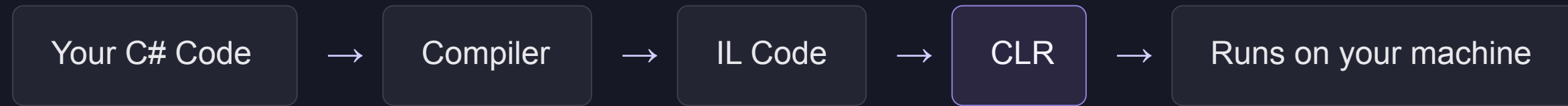
Shared type rules that let different .NET languages interoperate

06

Assemblies

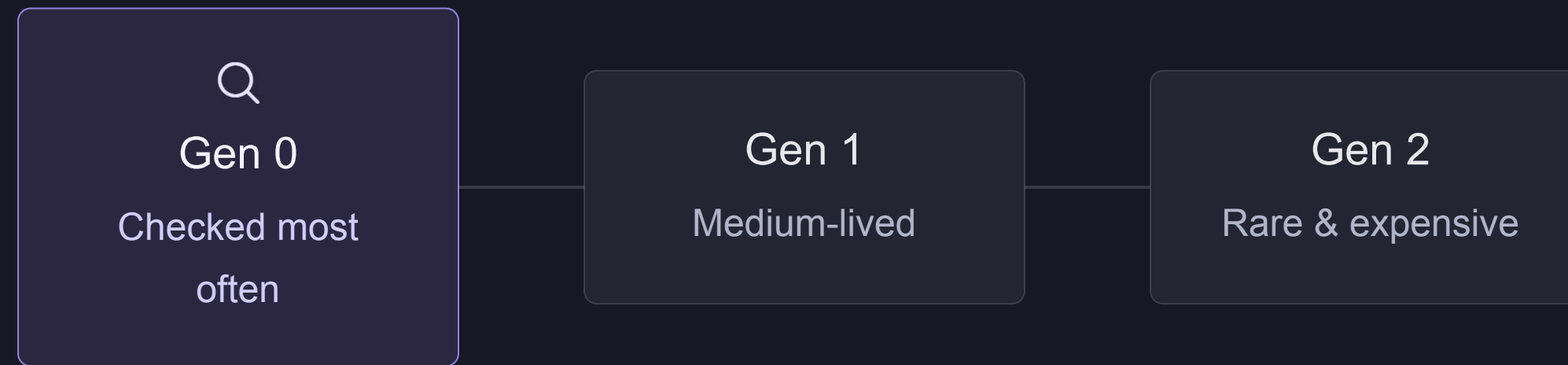
The compiled, deployable package your code ships as

CLR — The Engine Room



- Manages memory with the Garbage Collector
- Handles exceptions and errors
- Enforces security and type safety
- Converts IL code to native code via the JIT compiler

Garbage Collector — Automatic Memory Cleanup



- No manual `delete` — memory is freed automatically
- New objects start in Gen 0; survivors get promoted
- Process: mark (find live objects) → sweep (reclaim memory)

JIT Compiler — Translating IL to Native Code



IL is CPU-independent — the same compiled file runs on Windows, macOS, and Linux

BCL — Your Built-In Toolbox

Pre-built, tested, optimized code that ships with .NET

 Collections

 File I/O

 Networking

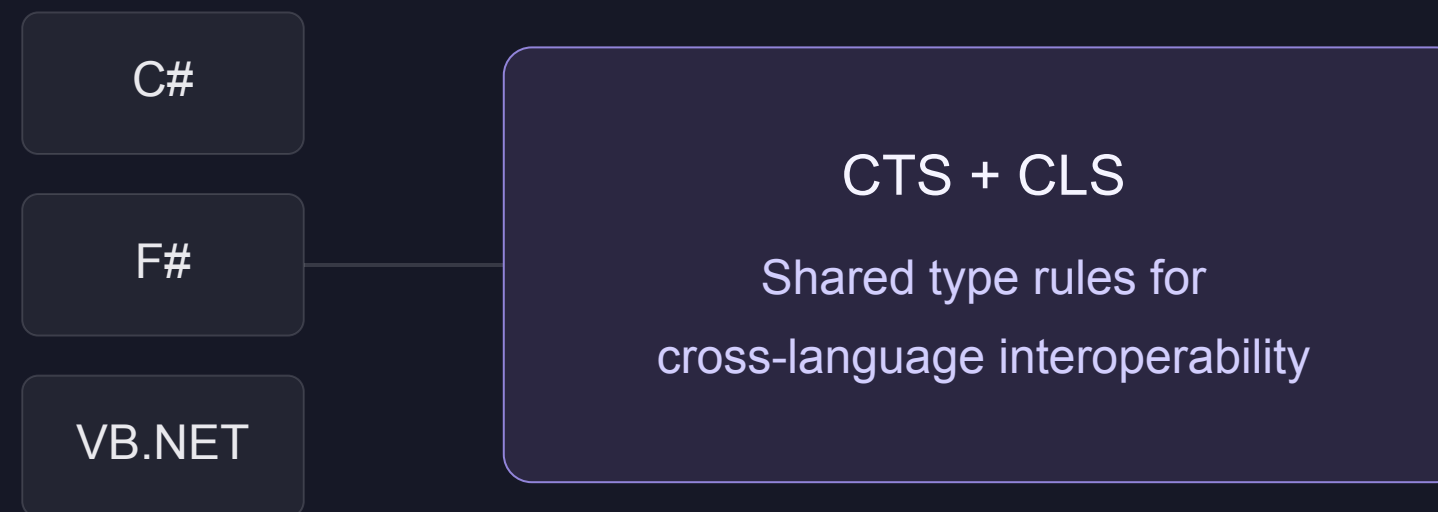
 LINQ

 String Utils

 Date / Time

FCL (Framework Class Library) is the older, broader term — most developers just say "BCL" today.

CTS & CLS — Making Languages Play Nice

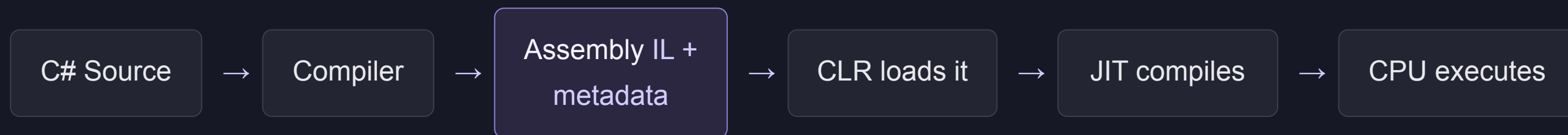


CTS — defines how types behave consistently across languages

CLS — rules that guarantee cross-language compatibility

Assemblies — The Final Package

- Assembly = the compiled output — a `.dll` or `.exe`
- Contains IL code + metadata (types, versions, references)
- Metadata lets the CLR understand your code's structure



Segment 04

Types of .NET Applications

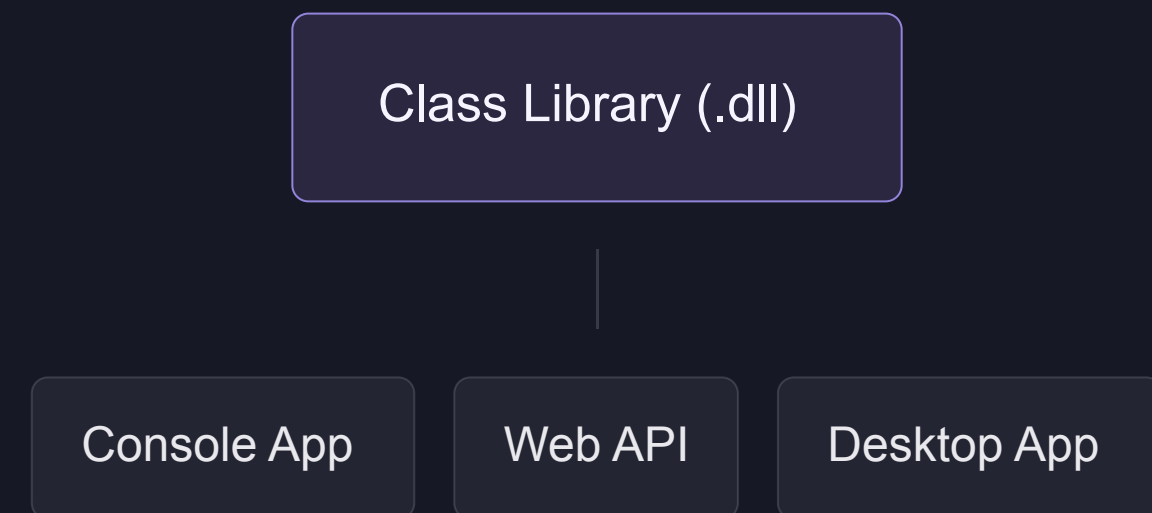
Console Application

- Simplest .NET app — no UI, runs in a terminal
- Great for learning, scripts, small tools, automation
- Usually the first project type beginners build



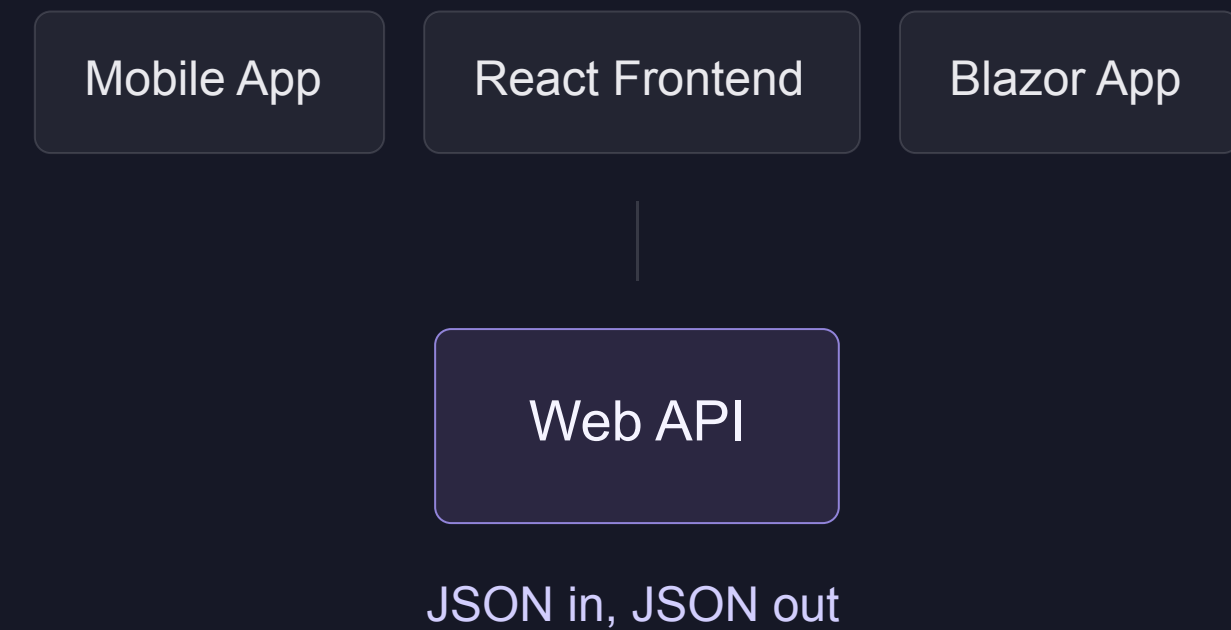
Class Library

- Not standalone — a reusable package referenced by other projects
- Compiles to a .dll, not an .exe
- Great for shared business logic and NuGet packages



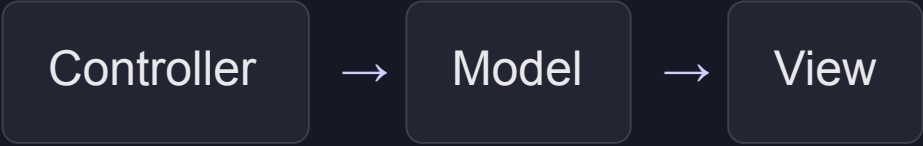
ASP.NET Core Web API

- Backend service — returns data (usually JSON), no UI
- Consumed by frontends, mobile apps, other services
- Backbone of modern cloud & microservices architecture



MVC vs Razor Pages

MVC



More structure — scales well for large, complex apps

Razor Pages

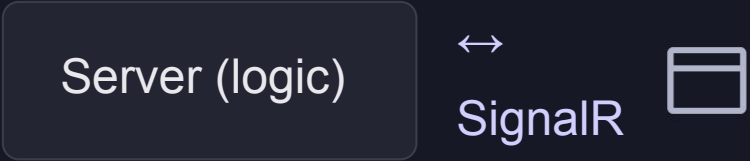


Simpler, page-focused — faster to start, less ceremony

Blazor — C# in the Browser

Build interactive web UI using C# instead of JavaScript

Blazor Server



Fast startup, needs a constant connection

Blazor WebAssembly



Larger initial download, no constant server connection

WinForms & WPF — Windows Desktop Apps

- **WinForms** — older, simpler, drag-and-drop designer
- **WPF** — newer, more powerful, XAML-based UI
- Both Windows-only, still widely used for internal tools



Windows only

Worker Service

- Long-running background process, no UI
- Great for scheduled jobs, queue processing, syncing
- Runs as a Windows Service or Linux daemon



Which Project Type Should I Use?

IF YOU NEED TO...	USE...
Build a quick script or learn the basics	Console Application
Share reusable code across projects	Class Library
Build a backend that returns data (JSON)	ASP.NET Core Web API
Build a full server-rendered website (complex)	ASP.NET Core MVC
Build a full server-rendered website (simple)	Razor Pages
Build an interactive web UI using C#	Blazor (Server or WASM)
Build a Windows-only desktop app	WinForms or WPF
Run a background/scheduled process	Worker Service

Segment 05

Q&A + Wrap-up

Thank you all!